

Deep Reinforcement Learning for Path-Following Control of an Autonomous Surface Vehicle using Domain Randomization

Tom Slawik, Bilal Wehbe, Leif Christensen, Frank Kirchner *

** All authors are with the German Research Center for Artificial Intelligence (DFKI GmbH), Robotics Innovation Center, Robert-Hooke-Straße 1, 28359 Bremen, Germany.
(e-mail: tom.slawik@dfki.de)*

Abstract: In this paper, we propose a path-following controller for an autonomous surface vehicle (ASV) that is based on model-free deep reinforcement learning. To make the learning agent more robust, we investigate domain randomization for sim-to-real transfer. We provide a comparison between three different algorithms: Deep Deterministic Policy Gradient (DDPG), Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO). The trained models are evaluated on the small-scale ASV Altus-LSA Niriis in the maritime test basin at DFKI RIC, Germany. Our results show that applying domain randomization leads to a significant performance improvement compared to no domain randomization, when tested on real hardware.

Copyright © 2024 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Keywords: Unmanned marine vehicles, Reinforcement learning control, Trajectory Tracking and Path Following, Sim-to-Real Transfer, Domain Randomization

1. INTRODUCTION

Unmanned surface vehicles are boats that can travel on the water surface without the need for a human operator on board. Typical applications include maneuvers that are dangerous for humans or time-consuming, tiresome operations. To make autonomous maneuvers possible, there is a rising demand for robust control architectures. Path-following is a motion strategy where the vehicle has to follow a given geometric curve as close as possible. Conventional approaches include PID-controllers (Fossen et al., 2003), LQR (Fossen, 2011), model predictive control (Zheng et al., 2014), and nonlinear control laws (Fossen, 2011). A model-predictive controller maintains a vehicle dynamics model and calculates the optimal actuator setpoints by solving a constrained optimization problem. The main disadvantage of this approach is that an exact representation of the target vehicle is required. Appropriate model assumptions have to be made beforehand, which may limit the achievable model performance. Moreover, it is hard to model uncertainties caused by unmodeled disturbances such as waves or wind.

Reinforcement learning aims to eliminate the need for such models by learning an optimal control policy from experience. Woo et al. (2019) proposed a path-following controller based on the deep deterministic policy gradient (DDPG) algorithm, which can handle continuous action spaces. The controller consists of a vector-field guidance that generates the desired course for every point on the water surface. The desired course is then fed into the

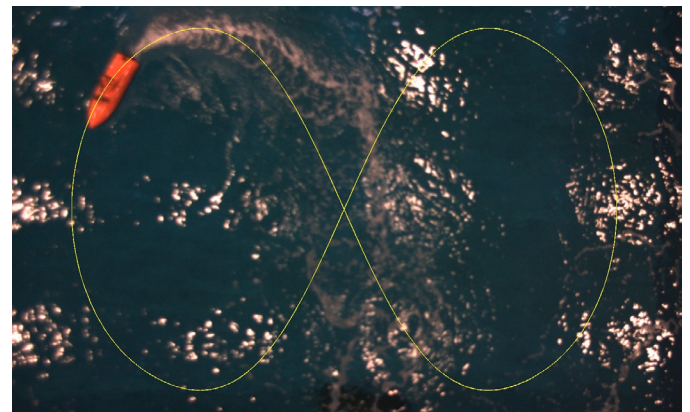


Fig. 1. Camera frame of the figure-eight path-following scenario in the test basin of DFKI RIC, Bremen, Germany.

deep learning algorithm to control the steering. Gonzalez-Garcia et al. (2020) proposed a similar approach, where high-level and low-level control are separated. Here, the guidance is performed by a deep reinforcement learning agent, while low-level control is performed by an adaptive sliding-mode controller.

In order to train a deep reinforcement learning agent, experiments have to be conducted to collect the required training data. Conducting these experiments on real hardware is time consuming and can cause safety issues during exploration. For this reason, training is usually performed in a simulated environment, followed by a transfer to the real system afterwards. However, there is a mismatch between simulation and reality, which leads to learning

* This work has received funding by the German Federal Ministry for Economic Affairs and Climate Action (grant No. 03SX540D).

a sub-optimal control strategy, which may perform worse when deployed to the real system. A more detailed survey on the simulation-to-reality gap was done by Zhao et al. (2020) for robotics in general. To date, there are only few experimental results quantifying the simulation-to-reality gap for motion control of marine vehicles. This paper aims to provide empirical data on how deep reinforcement learning algorithms perform on real hardware by deploying a RL-based path-following agent on the small-scale ASV Altus Niriis. The experiments are conducted at the maritime hall of the DFKI RIC in Bremen, Germany, maintaining a 23 m × 19 m test basin for maritime robotics (DFKI Robotics Innovation Center, 2022). We propose a path-following agent that comprises a novel state vector embedding information on the upcoming path. To be able to follow arbitrary curved and rectilinear paths, the agent is trained using a custom sampling algorithm generating different training paths. Domain randomization is a technique where the simulation model parameters are perturbed at given time intervals to reduce overfitting in order to make the agent more robust against model uncertainties and disturbances (Josifovski et al., 2022). We compare three different learning algorithms: DDPG (Deep Deterministic Policy Gradient) (Lillicrap et al., 2016), SAC (Soft Actor-Critic) (Haarnoja et al., 2018) and PPO (Proximal Policy Optimization) (Schulman et al., 2017) and how they perform with different randomization parameters.

First, the simulation environment needed for the training procedure is described. In the next section, the Markov decision process and the learning algorithm are specified. The trained models are then evaluated on the real hardware. Finally, the obtained results are discussed.

2. PROBLEM FORMULATION AND METHODOLOGY

2.1 Path-following problem

Let $\mathbf{p}_d(\xi) = (x_d(\xi), y_d(\xi)) \in \mathbb{R}^2$ be a parameterized curve describing the desired vehicle path, where $\xi \in [a, b]$ is a continuous scalar variable. To simplify the calculations, we approximate the reference path $\mathbf{p}_d(\xi)$ by a polygon $\mathcal{P} = (x_0, y_0, \dots, x_N, y_N)$ with a segment length of 0.1 m.

The cross-track error is defined as the perpendicular distance of the vehicle relative to the reference path, and can be calculated using a coordinate transform:

$$e_y(t) = -(x(t) - x_s) \cos(\alpha_s) + (y(t) - y_s) \sin(\alpha_s). \quad (1)$$

Here, $s \in [0..N]$ is an index variable that represents the current polygon segment that the vehicle is guided along. α_s is the orientation of this segment relative to the world axes:

$$\alpha_s = \text{atan2}(y_{s+1} - y_s, x_{s+1} - x_s) \quad (2)$$

The goal of path-following control is to move along the desired curve, while minimizing the cross-track error $e_y(t)$ (Skjetne et al., 2004a). A real-world example is given in Fig. 1, where the figure-eight shape outlines the reference path. In contrast to trajectory tracking, path-following does not define temporal constraints.

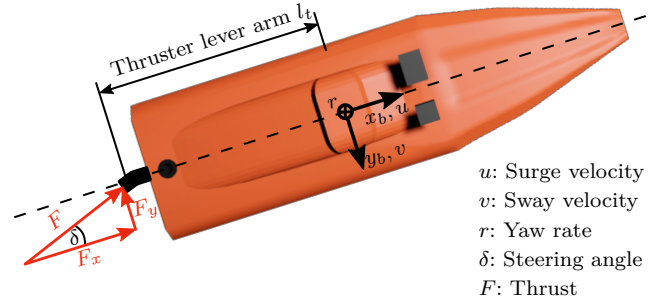


Fig. 2. Illustration of the modeled vehicle kinematics and propulsive forces.

2.2 Vehicle Dynamics

The considered vehicle is modeled in three degrees of freedom (DOF): u is the surge velocity (longitudinal), v is the sway velocity (sideways), and r is the angular velocity, all summarized by the velocity vector $\mathbf{v} = (u, v, r)^\top$ in a body-fixed frame. The current position of the vehicle on the water plane follows the NED (north-east-down) convention and is given by $\boldsymbol{\eta} = (x, y, \psi)^\top$, where ψ is the heading angle. The vehicle dynamics are modeled as a nonlinear 3 DOF model according to Fossen (2011):

$$\underbrace{\mathbf{M}_{RB}\dot{\mathbf{v}} + \mathbf{C}_{RB}(\mathbf{v})\mathbf{v}}_{\text{rigid-body term}} + \underbrace{\mathbf{M}_A\dot{\mathbf{v}} + \mathbf{C}_A(\mathbf{v})\mathbf{v} + \mathbf{d}(\mathbf{v})}_{\text{hydrodynamic term}} = \boldsymbol{\tau}.$$

The above dynamics equation splits the system into two separate parts: First, there is the rigid body, modeled by the mass matrix $\mathbf{M}_{RB}$, and the coriolis-centripetal matrix $\mathbf{C}_{RB}(\mathbf{v})$. The second part models the interaction between the rigid body and the surrounding fluid using the mass matrix $\mathbf{M}_A$, the coriolis-centripetal matrix $\mathbf{C}_A$, and the nonlinear damping term $\mathbf{d}(\mathbf{v})$. The hydrodynamic damping represents an energy dissipation caused by the surrounding fluid. It is modeled using the second-order modulus functions proposed by Blanke (1981):

$$\mathbf{d}(\mathbf{v}) = (\mathbf{D}_{lin} + \mathbf{D}_{nonlin}(\mathbf{v}))\mathbf{v}. \quad (3)$$

To solve the system equations, Euler's forward method with a constant time step of $\Delta t = 0.1$ s is used (Atkinson, 1989). As it is often the case for marine vehicles, the hydrodynamic coefficients of the target vehicle are not known in advance and are hard to identify. For this reason, we adopt the hydrodynamic parameters of the model ship CyberShip II from the Marine Cybernetics Laboratory, Trondheim. The vehicle has a mass of $m = 23.8$ kg and is 1.255 meters long, while its breadth is 0.29 meters (Skjetne et al., 2004b). For reference, the parameters of this vehicle are listed in appendix Table 4.

2.3 Propulsion

The target vehicle is propelled by a hydrojet propulsion system, which is located at the middle of the stern. The resulting force vector is:

$$\boldsymbol{\tau}(F(t), \delta(t)) = \begin{pmatrix} F(t) \cos(\delta(t)) \\ -F(t) \sin(\delta(t)) \\ -F(t) l_t \sin(\delta(t)) \end{pmatrix}. \quad (4)$$

Here, $\delta(t) \in [-16^\circ, +16^\circ]$ is the steering angle, and $F(t) \in [0 \text{ N}, 35 \text{ N}]$ is the propulsive force. l_t is the distance

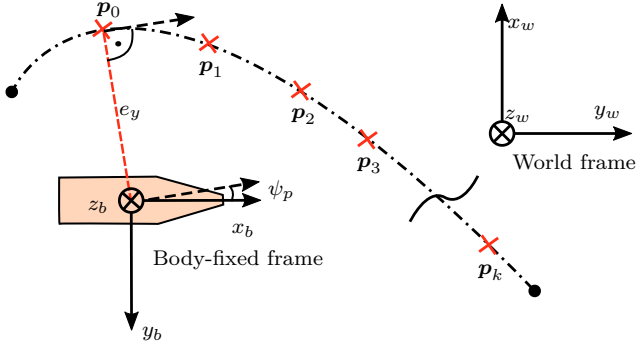


Fig. 3. The reference path is sampled at equidistant points, which are then fed into the observation vector.

between the nozzle and the origin of the vehicle. The trigonometrical relationships to derive the resulting force / torque vector are illustrated in Fig. 2.

3. MARKOV DECISION PROCESS

The control process is modeled as a Markov decision process, which is represented by a series of discrete time steps. At each time step t , the system dynamics are described by a state vector $\mathbf{s}_t$. Based on the state vector, the decision-making agent generates an action $\mathbf{a}_t$ to control the system.

3.1 Action Vector

During the control process, only the steering δ of the vehicle is controlled. For this reason, the action vector is of dimension one:

$$\mathbf{a}_t = (\delta) ,$$

where δ is normalized to $[-1.0, 1.0]$.

3.2 State Vector

The purpose of the state vector is to provide the algorithm with all the information necessary to control the system. The proposed state vector is defined as follows:

$$\mathbf{s}_t = [u(t), v(t), r(t), \delta(t), \psi_p(t), \tilde{\mathbf{p}}(t)]^\top . \quad (5)$$

$u(t)$, $v(t)$ and $r(t)$ are the surge, sway and yaw velocities, respectively. $\delta(t)$ is the current steering angle, normalized to $[-1.0, +1.0]$. $\psi_p(t)$ denotes the current heading angle defined relative to the path, denoted in radians. All length-related values are denoted in the metric system: m for positional data, and m/s for velocities.

In order to allow for predictive steering, the path geometry of the upcoming route is embedded into a vector $\tilde{\mathbf{p}}(t)$, containing 7 discretized two-dimensional points with a step size of 1 m along track:

$$\tilde{\mathbf{p}}(t) = [p_{0,x}, p_{0,y}, \dots, p_{k,x}, p_{k,y}]^\top \in \mathbb{R}^{2k+2} . \quad (6)$$

To avoid learning a global representation of world coordinates, the discretized points are described relative to the body-fixed coordinate system. The first sample $(p_{0,x}(t), p_{0,y}(t))$ is based on the orthogonal projection of the vehicle onto the path (as illustrated in Fig. 3).

3.3 Reward Function

The reward signal serves as a feedback for the algorithm to model the desired goal behavior. For path-following control, the overall goal is to drive along a predefined path in a given direction while minimizing the resulting cross-track error. In this process, it must be ensured that the actuation mechanics are not subject to too much stress. These requirements result in three sub goals that are formulated as a sum of three separate reward functions: A tracking reward r_e , an orientation reward r_h , and an actuation reward r_{σ_δ} :

$$r = w_e r_e + w_h r_h + w_{\sigma_\delta} r_{\sigma_\delta} , \quad (7)$$

where w_e , w_h and w_{σ_δ} are weighting factors for the tracking reward, the orientation reward and the actuation reward, respectively.

Tracking Reward

The tracking reward evaluates the cross-track error for the current time step and is defined as an exponential function:

$$r_e = e^{-k_1 |e_y|} , \quad (8)$$

where k_1 is a fixed constant describing the slope of the reward function. If the error is zero, the partial reward reaches its maximum of 1.0.

Orientation Reward

By defining the tracking reward alone, the learner has no indication of which direction to drive along the path. For this reason, an orientation reward is proposed that becomes negative when driving in the opposite direction and remains at zero when driving in the correct direction:

$$r_h = \begin{cases} -1 & \text{if } |\psi_p| > \frac{\pi}{2} \\ 0 & \text{otherwise} \end{cases} . \quad (9)$$

Actuation Reward

By default, the proposed reinforcement learning algorithm generates an output signal that is not constrained regarding smoothness and rate of change. This can lead to chattering effects, which negatively affect actuators and cause wear or even damage to the mechanics. For this reason, an actuation reward is implemented to smoothen the generated control signal. Woo et al. (2019) proposed a formalism where the standard deviation of the output signal is used as input for an exponential function:

$$r_{\sigma_\delta} = e^{-k_2 \sigma_\delta} , \quad (10)$$

where k_2 is a constant describing the decay of the exponential function. The standard deviation σ_δ takes the steering values from the preceding 2 seconds into account, which, for a time step of $dt = 0.1$ s is the running standard deviation of the last $n_\delta = 20$ time steps, starting from the current time step k :

$$\sigma_\delta = \sqrt{\frac{1}{n_\delta} \sum_{i=0}^{n_\delta-1} (\delta_{k-i} - \bar{\delta})^2} \quad \text{with} \quad \bar{\delta} = \frac{1}{n_\delta} \sum_{i=0}^{n_\delta-1} \delta_{k-i} . \quad (11)$$

The required reward function weights and coefficients were chosen experimentally using random-search and are set to: $w_e = 0.2$, $w_h = 0.8$, $w_{\sigma_\delta} = 0.9$, $k_1 = 1.0$, and $k_2 = 3.0$.

Algorithm 1 Generate random path

```

procedure SAMPLE_PATH( $N, \sigma_\alpha, l_{\min}, l_{\max}$ )
   $x_0 \sim \mathcal{U}(x_{\min}, x_{\max})$ 
   $y_0 \sim \mathcal{U}(y_{\min}, y_{\max})$ 
  for  $k \leftarrow 0$  to  $N$  do
    repeat ▷ If sampled segment is outside workspace
      if  $k = 0$  then
         $\alpha_0 \sim \mathcal{U}(-\pi, +\pi)$  ▷ Initial direction
      else
         $\Delta\alpha \sim \mathcal{N}(0, \sigma_\alpha)$  ▷ Deflection of segment
         $\alpha_k \leftarrow \alpha_{k-1} + \Delta\alpha$ 
      end if
       $l_k \sim \mathcal{U}(l_{\min}, l_{\max})$  ▷ Length of segment
       $x_{k+1} \leftarrow x_k + l_k \cos(\alpha_k)$ 
       $y_{k+1} \leftarrow y_k + l_k \sin(\alpha_k)$ 
    until  $x_{\min} < x_{k+1} < x_{\max}$ 
      and  $y_{\min} < y_{k+1} < y_{\max}$ 
    end for
  return  $(x_0, y_0, x_1, y_1, \dots, x_N, y_N)$ 
end procedure

```

4. LEARNING

The training procedure entirely takes place in the simulated environment described in section 2. Main component of the proposed learning procedure is a sampling algorithm that successively samples a random path the vehicle has to follow. The agent tries to follow the sampled path as close as possible with a constant throttle value n , which is sampled uniformly from $[0.2, 1.0]$ for each episode. As soon as the path-following maneuver is finished, the training episode ends and a new episode starts. To prevent the algorithm from getting into an endless loop when the vehicle loses its track, the maximum number of time steps during each episode is limited to 1000. At the beginning of every training episode, the vehicle is placed exactly on the starting position of the desired path and aligns with it. Additionally, the simulation model parameters are randomized to improve the agent's robustness to changing dynamics.

4.1 Path Sampling Algorithm

The proposed path sampling algorithm iteratively generates N individual path segments, which are constrained to the workspace $\mathcal{W} = (x_{\min}; x_{\max}) \times (y_{\min}; y_{\max})$. The initial position and direction of the path are sampled from a uniform distribution. Then, for each path segment, a length l_k and a deflection $\Delta\alpha$ is sampled. The deflection is the change of direction relative to the previous path segment and follows a Gaussian normal distribution $\mathcal{N}(0, \sigma_\alpha)$ with variance σ_α and $\mu_\alpha = 0$. The second parameter of the segment is its length l_k . In contrast to the deflection, here, a uniform distribution $\mathcal{U}(l_{\min}, l_{\max})$ is used. After a new path segment is sampled, the segment's endpoint (x_{k+1}, y_{k+1}) is verified to lie within the workspace $\mathcal{W}$. If the endpoint exceeds the workspace, the line segment is discarded and a new path segment is generated. The detailed algorithm is outlined in Algorithm 1, while the parameters of the path-sampling algorithm are listed in

Table 1. Parameters used for the path-sampling algorithm.

Parameter	Value
Workspace	$\mathcal{W} = (2 \text{ m}; 17 \text{ m}) \times (2 \text{ m}; 21 \text{ m})$
Number of segments	$N = 20$
Minimum segment length	$l_{\min} = 2.0 \text{ m}$
Maximum segment length	$l_{\max} = 6.0 \text{ m}$
Deflection variance	$\sigma_\alpha = 0.5 \text{ rad}$

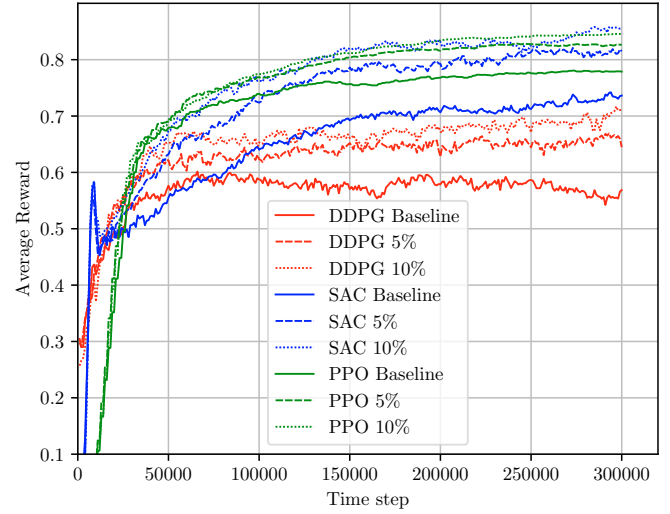


Fig. 4. Average reward of the trained agents, evaluated every 1000 time steps in a separate simulation environment.

Table 1. After a path is generated, the resulting nodes are used as nodes for a cubic spline to get a curved path.

4.2 Domain Randomization

Let $\theta \in \mathbb{R}^{25}$ be a parameter vector specifying the model parameters of a single vehicle model. The sampling procedure consists of a random vector generator that applies Gaussian noise to the baseline model θ_0 :

$$\theta \sim \mathcal{N}(\theta_0, \Sigma). \quad (12)$$

The covariance matrix Σ is chosen to generate uncorrelated noise that is scaled to the corresponding parameters:

$$\Sigma = \beta \text{diag}(\theta_0). \quad (13)$$

Here, $\beta \in [0.0, 1.0]$ is the fraction of noise applied to the baseline model, and $\text{diag}(\theta_0)$ is a diagonal matrix with the entries of θ_0 on the diagonal.

Sampling a set of model parameters according to (12) does not necessarily generate a valid dynamics model. For example, the system dynamics might oscillate or lead to a numerical overflow error. In order to identify these faulty models, each model candidate is automatically simulated and tested beforehand, where each vehicle performs a turning maneuver. If the dynamics start to oscillate or raise an overflow error, the model candidate is considered invalid, and a new model candidate is sampled.

4.3 Training

During training, we randomize the original vehicle model parameters by $\beta = 0\%$, $\beta = 5\%$, and $\beta = 10\%$ at

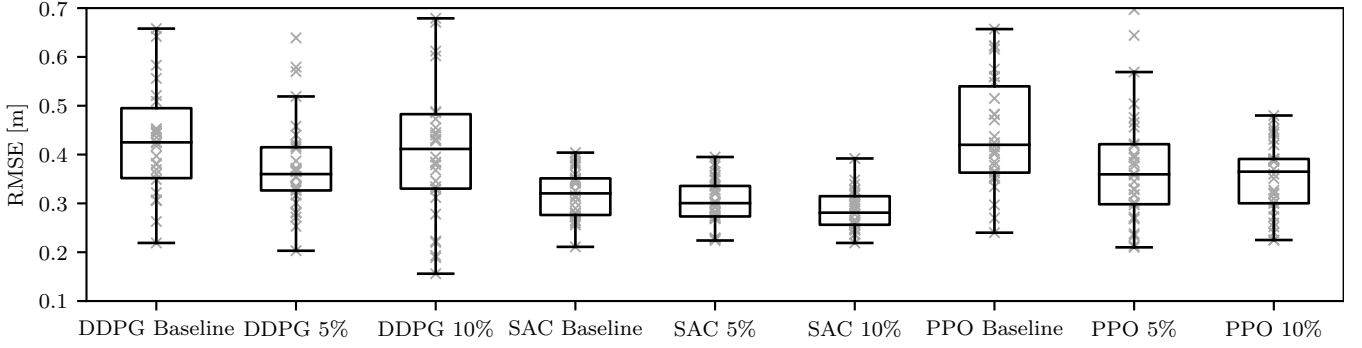


Fig. 5. Box-Plot of the real-world tracking results: Each dot represents the RMS tracking error of one lap of the figure-eight scenario (see Fig. 1). Each box contains 50 % of the data, and its horizontal line marks the median value.

Table 2. Training hyperparameters (DDPG)

Parameter	Value
Total time steps	300 000
Discount factor γ	0.9
Polyak-coefficient τ	0.02
Learning rates	$\alpha = 0.0004, \beta = 0.0004$
Critic-network	160×96 neurons, tanh activation function
Actor-network	256×64 neurons, tanh activation function
Batch size	128
Replay buffer size	10 000
Training frequency	64
Action noise	0.1 normal noise

each episode. To find the optimal hyperparameters for training, we did a random-search on a non-randomized DDPG agent. The resulting hyperparameters are listed in Table 2. For the SAC and PPO algorithms, we use the exact same discount factor, Polyak-coefficient, batch size, neural networks, and learning rates. Since the performance of a trained agent is highly dependent on randomness, we train 100 policies per configuration using different random seeds. The performance is evaluated every 1000 time steps in a separate simulation environment, which maintains a different test vehicle model (see Table 4) to imitate the transfer to another system (sim-2-sim transfer). The test takes place on a figure-eight shape with constant throttle ($n = 1.0$) with the exact same conditions for each pass. The resulting progression can be seen in Fig. 4, which shows the average reward obtained in the simulated testing environment for different configurations.

5. EVALUATION - REAL HARDWARE

5.1 Experimental Setup

To evaluate the trained policies on real hardware, we conducted a series of experiments on the small-scale ASV Niriis from Altus-LSA, which has a length of 135 cm, a breadth of 38 cm, and a weight of 10.5 kg. The pose of the vehicle is estimated by a calibrated RGB camera (2048×2048 px @ 26 fps), which is attached to the ceiling. To perform the neural network predictions, a Raspberry Pi 4B was mounted inside the boat, invoking the policy with a frequency of 100 Hz. In parallel it also runs a customized software stack, which includes an extended Kalman filter to fuse the detections coming from the ceiling camera with the inertial measurement unit InvenSense MPU-9250.

Table 3. Experimental Results: The SAC algorithm with 10 % randomization yielded the minimum tracking error.

Algo.	Noise	RMSE (Real hardware)	RMSE (Simulation)
DDPG	None	$0.437 \text{ m} \pm 0.129 \text{ m}$	$0.226 \text{ m} \pm 0.008 \text{ m}$
DDPG	5%	$0.377 \text{ m} \pm 0.098 \text{ m}$	$0.133 \text{ m} \pm 0.017 \text{ m}$
DDPG	10%	$0.419 \text{ m} \pm 0.156 \text{ m}$	$0.123 \text{ m} \pm 0.016 \text{ m}$
SAC	None	$0.319 \text{ m} \pm 0.050 \text{ m}$	$0.180 \text{ m} \pm 0.016 \text{ m}$
SAC	5%	$0.305 \text{ m} \pm 0.047 \text{ m}$	$0.096 \text{ m} \pm 0.011 \text{ m}$
SAC	10%	$0.285 \text{ m} \pm 0.040 \text{ m}$	$0.076 \text{ m} \pm 0.004 \text{ m}$
PPO	None	$0.444 \text{ m} \pm 0.111 \text{ m}$	$0.185 \text{ m} \pm 0.011 \text{ m}$
PPO	5%	$0.375 \text{ m} \pm 0.119 \text{ m}$	$0.157 \text{ m} \pm 0.009 \text{ m}$
PPO	10%	$0.353 \text{ m} \pm 0.075 \text{ m}$	$0.140 \text{ m} \pm 0.009 \text{ m}$

However, we noticed the output of the MPU-9250 to be very noisy, as it is a very cheap model. Unfortunately, the time derivative of the computer vision position estimate was noisy as well. This resulted in a very noisy velocity estimate, varying around 20 %. For this reason, we did not control the velocity of the vehicle. Instead, we have chosen a constant throttle of 40 %, resulting in a maximum velocity of 1.5 m/s. Since our path-following controller also relies on the velocity estimate, we consider the noise as a disturbance, verifying the robustness of the controller.

For each configuration, we conducted a test series with ten different policies, selected according to the best-performing random seeds in simulation. For testing, we were using the figure-eight shape shown in Fig. 1 and performed a path-following maneuver for three laps (+1 warm-up lap to get a consistent starting position).

5.2 Experimental Results

The results of our experiments are illustrated in Fig. 5, showing a box-plot where each point represents the RMS tracking error of one lap (excluding warm-up laps). Each box contains 50 % of the obtained data, which scatters around the median (outlined as a horizontal line). A summary of these results is given in Table 3, which also contains the results obtained in simulation, for reference.

From the data it can be seen that both the DDPG and PPO baselines, which are trained without domain randomization, perform the worst in both simulation and reality for the given task. Applying the domain randomization technique described in section 4.2 with an amount of 5 % Gaussian noise led to a reduction of the tracking error by a

substantial amount for both DDPG and PPO. Increasing the amount of noise to 10 percent further improved the performance of PPO by a minimal amount while improving robustness. For DDPG, however, 10 % domain randomization has shown a somehow contradicting behavior. There was one policy yielding a tracking error below 0.2 meters, which is the best of all results. However, most of the policies were resulting in a very bad performance. We could also observe that this amount of randomization on DDPG increased the chance of unexpected actions, such as performing a sudden left/right turn or crashing into the walls. The most robust tracking results were obtained using the SAC algorithm. Even without randomization, SAC on average outperformed both the randomized DDPG and PPO models. We found that using domain randomization on SAC merely led to minor improvements in our setting, even though it had a huge effect in simulation.

6. CONCLUSION

In this work we evaluated domain randomization for a deep reinforcement learning based path-following controller that is capable of arbitrary curved paths. After training the agent in simulation, a series of experiments were carried out to investigate the transfer of the learned policy from simulation onto a real system. We compared three different algorithms: DDPG, SAC and PPO. Our experimental results show that domain randomization was in general able to improve the tracking error of the tested agents. In particular, PPO benefited the most from applying randomization, where the tracking error was significantly reduced. DDPG on the other hand showed improvement with 5 % randomization, however further increase of the randomization noise deteriorated its performance. SAC showed also a slight improvement with adding randomization noise, showing in general the best tracking performance among the tested policies, and maintaining maximum robustness. One possible explanation for this might lie in the architecture of SAC itself: SAC is based on an entropy-maximization framework, which aims to maximize the stochastic variance of the control action during exploration, hence reducing the chance of remaining in local minima.

The results of this work can be used as a baseline for further investigations. While this work relies on an offline-learning approach, the authors are planning to investigate some online-learning approaches as well where the vehicle would adapt to changing dynamics during operation.

REFERENCES

- Atkinson, K.E. (1989). *An Introduction to Numerical Analysis*. John Wiley & Sons, New York, second edition.
- Blanke, M. (1981). Ship propulsion losses related to automatic steering and prime mover control.
- DFKI Robotics Innovation Center (2022). Maritime exploration hall. URL <https://robotik.dfk-bremen.de/en/>.
- Fossen, T., Breivik, M., and Skjetne, R. (2003). Line-of-sight path following of underactuated marine craft.
- Fossen, T.I. (2011). *Handbook of marine craft hydrodynamics and motion control*. John Wiley & Sons.

- Gonzalez-Garcia, A., Castañeda, H., and Garrido, L. (2020). Usv path-following control based on deep reinforcement learning and adaptive control. In *Global Oceans 2020: Singapore – U.S. Gulf Coast*, 1–7.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In J.G. Dy and A. Krause (eds.), *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10–15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, 1856–1865. PMLR.
- Josifovski, J., Malmir, M., Klarmann, N., Zagar, B.L., Navarro-Guerrero, N., and Knoll, A. (2022). Analysis of Randomization Effects on Sim2Real Transfer in Reinforcement Learning for Robotic Manipulation Tasks. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 10193–10200. IEEE, Kyoto, Japan.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2016). Continuous control with deep reinforcement learning. In *ICLR*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, abs/1707.06347.
- Skjetne, R., Fossen, T., and Kokotović, P. (2004a). Robust output maneuvering for a class of nonlinear systems. *Automatica*, 40.
- Skjetne, R., Smogeli, O., and Fossen, T. (2004b). Modeling, identification, and adaptive maneuvering of cyber-ship ii: A complete design with experiments.
- Woo, J., Yu, C., and Kim, N. (2019). Deep reinforcement learning-based controller for path following of an unmanned surface vehicle. *Ocean Engineering*, 183.
- Zhao, W., Queralta, J.P., and Westerlund, T. (2020). Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 737–744.
- Zheng, H., Negenborn, R.R., and Lodewijks, G. (2014). Trajectory tracking of autonomous vessels using model predictive control. *IFAC Proceedings Volumes*, 47(3), 8812–8818. 19th IFAC World Congress.

APPENDIX

6.1 Model Parameters

Table 4. Model parameters of the vehicle model used for training and evaluation

Param.	Training	Test	Param.	Training	Test
m	23.8	20.1	$N_{\dot{r}}$	−1.0	−1.01
I_z	1.760	2.03	$X_{ u u}$	−1.32742	−1.49
x_g	0.046	0.05	$Y_{ v v}$	−36.47287	−43.94
y_g	0.0	0.0	$Y_{ v r}$	−0.845	−0.85
X_u	−0.72253	−0.63	$Y_{ r v}$	−0.805	−0.75
Y_v	−0.88965	−0.85	$Y_{ r r}$	−3.450	−3.91
Y_r	−7.250	−7.17	$N_{ v v}$	3.95645	4.55
N_v	0.03130	0.03	$N_{ r v}$	0.130	0.12
N_r	−1.900	−1.88	$N_{ v r}$	0.080	0.07
$X_{\dot{u}}$	−2.0	−2.02	$N_{ r r}$	−0.750	−0.92
$Y_{\dot{v}}$	−10	−11.19	X_{uuu}	−5.86643	−6.41
$Y_{\dot{r}}$	−0.0	0.0	l_t	−0.6 m	−0.6 m
$N_{\dot{\theta}}$	−0.0	0.0			